

Claude Code

generación de código con criterio

Spec-Driven Development para equipos técnicos

Lo que vamos a ver hoy

- 01 **Repaso** · de la sesión 1
- 02 **Por qué el "vibe coding" se rompe** · a partir de cierto tamaño
- 03 **Qué es Spec-Driven Development** · y por qué es la metodología de referencia en 2026
- 04 **Las 4 fases del flujo SDD** · Specify → Plan → Tasks → Implement
- 05 **SDD "a mano" en Claude Code** · con slash commands propios
- 06 **SDD con tooling** · GitHub Spec Kit, Kiro, otras opciones
- 07 **Mantener la spec viva** · spec drift, regeneración, validación cruzada
- 08 **Cierre, deberes y preguntas**

MÓDULO 01

Repaso:

qué hicimos, qué tocaba hacer

Tres ideas que cerraron la sesión 1

01 Memoria explícita en archivos

CLAUDE.md + auto-memoria en ~/.claude/projects/. Lo que está en Git, es del equipo.

02 Empieza minimalista

CLAUDE.md + settings.json + 1–2 commands. Ya tienes el 80% del valor.

03 Explore → Plan → Code → Commit

Plan Mode (Shift+Tab×2) y editar el plan con Ctrl+G: las dos teclas más usadas.

Lo que teníais que traer hecho

-  01 Crear la estructura `.claude/` en un proyecto vuestro
-  02 Escribir un `CLAUDE.md` con las 5 secciones (Stack, Comandos, Convenciones, Decisiones, Anti-patronos)
-  03 Crear vuestro primer slash command propio
-  04 Hacer al menos un ciclo completo Explore → Plan → Code → Commit en una tarea real
-  05 Traer dos preguntas concretas sobre algo que no funcionó

CONEXIÓN CON LA SESIÓN DE HOY

Explore → Plan → Code → Commit

era la versión simple

Hoy lo formalizamos.

SDD versiona el "Plan" en artefactos y convierte la spec en la fuente de verdad. No el código.

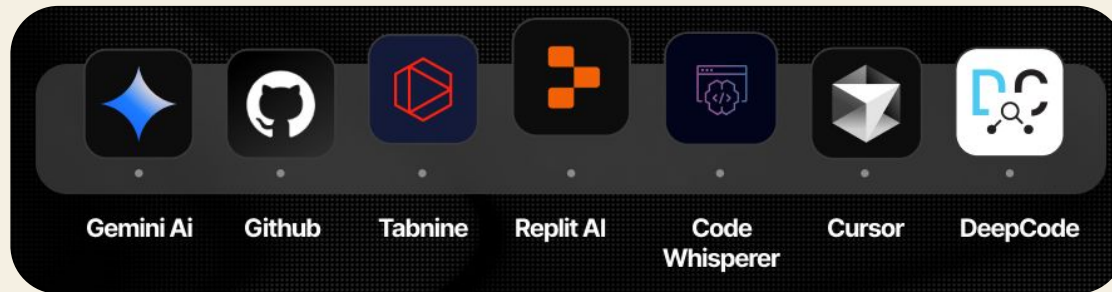
MÓDULO 02

El problema:

por qué el vibe coding se rompe

Qué es "vibe coding"

Pedirle a un agente código por intuición, en lenguaje natural, sin estructura previa. Iterar hasta que "funciona".



Funciona genial para prototipos. Se cae en producción.

EL TÉRMINO

Lo popularizó Andrej Karpathy a principios de 2025.

Llevamos más de un año largo viendo dónde se cae.

Lo que dicen los benchmarks y el trabajo de campo

9.8–42.1%

tasa de código
vulnerable generado
por LLMs

Yan et al., 2025

110.000+

issues introducidos
por IA viviendo en
repos productivos

arXiv, feb 2026

10×

menos "regenerar
desde cero" con flujo
SDD vs ad-hoc

GitHub, 2025-2026

40h → 8h

features
documentadas con
spec-first (Kiro)

AWS Kiro, 2026

Vibe coding vs SDD

VIBE CODING

- ✗ Pasan los unit tests
- ✗ Viola un patrón arquitectónico
- ✗ Rompe un contrato de API que nadie escribió
- ✗ Mete un anti-patrón de seguridad que aflora en producción

"El test pasa, pero algo está mal y nadie sabe qué."

SPEC-DRIVEN DEVELOPMENT

- ✓ La spec define el contrato
- ✓ El plan define la arquitectura
- ✓ Las tareas definen el orden
- ✓ El código es solo el output, regenerable

"La spec es la fuente de verdad. El código es desechable."

LA FRASE QUE LO RESUME

"The spec
is the prompt."

La spec, escrita con suficiente detalle, es el prompt.

Los agentes son cada vez más capaces; lo que sigue siendo escaso es la claridad.

MÓDULO 03

Qué es *Spec-Driven Development*

SDD en una frase

LA IDEA CLAVE

Una metodología en la que una *especificación ejecutable y versionada en control de código* es la única fuente de verdad. El equipo (o un agente) escribe primero una spec detallada, deriva un plan, lo trocea en tareas atómicas, y solo entonces genera código. Si los requisitos cambian, se edita la spec y se regenera lo que toca.

https://es.wikipedia.org/wiki/Desarrollo_dirigido_por_especificaciones

Antes · con SDD

ANTES

Código = fuente de verdad. La doc se queda obsoleta.

Cambias el código, intentas actualizar la doc.

Conocimiento en cabezas y wikis dispersas.

Implementación = lo caro.

CON SDD

Spec = fuente de verdad. El código es output regenerable.

Cambias la spec, regeneras el código afectado.

Conocimiento en specs versionadas que el agente lee.

Implementación = barata. Lo escaso es la claridad.

Línea temporal corta

2025 · Q1

Karpathy populariza "vibe coding". Funciona para prototipos.

2025 · Q2-Q3

Primeros experimentos formales: equipos escriben spec antes de prompt.

2025 · Sep

GitHub abre Spec Kit. AWS lanza Kiro.

2026 · Ene

Paper fundacional arXiv: "Spec-Driven Development: From Code to Contract".

2026 · Q1-Q2

Spec Kit pasa de 70k stars. DeepLearning.AI lanza curso oficial. SDD es mainstream.

SDD vs TDD vs BDD vs PRD vs DDD

PRÁCTICA	QUÉ DEFINE	RELACIÓN CON SDD
TDD Test driven developer	Cómo se comporta a nivel de unidad	<i>Complementaria · las tareas SDD pueden exigir TDD dentro</i>
BDD Behavior-Driven Development	Escenarios de usuario observables	<i>SDD da los invariantes estructurales que BDD asume</i>
PRD Product Requirements Document	Documento para humanos	<i>SDD es el equivalente ejecutable y vivo</i>
DDD Domain-Driven Design	Modelado del dominio	<i>SDD puede materializar la ubiquitous language en spec</i>

MÓDULO 04

Las cuatro fases

Specify → Plan → Tasks → Implement

Cuatro fases

Cada fase produce un artefacto que alimenta a la siguiente.

No se pasa de fase sin validar la anterior.

FASE 01

Specify

Qué se construye y por qué.
User journeys, criterios de éxito.
Sin stack técnico.

FASE 02

Plan

Stack, arquitectura, decisiones técnicas.
Por fases: Foundation, Core, Polish.

FASE 03

Tasks

Trocear el plan en tareas atómicas con dependencias y archivos.

FASE 04

Implement

El agente ejecuta tareas.
El humano revisa cambios pequeños y enfocados.

Qué se construye, por qué — sin stack

PRODUCE

spec.md

- ✓ User stories y journeys
- ✓ Requisitos funcionales
- ✓ Criterios de éxito medibles
- ✓ Casos frontera identificados

NO CONTIENE

- ✗ Tecnologías concretas
- ✗ Estructura de carpetas
- ✗ Diseño de base de datos

EJEMPLO DE PROMPT

Construye una aplicación que me ayude a organizar mis fotos en álbumes separados.

Los álbumes se agrupan por fecha y se pueden reorganizar arrastrando en la página principal.

Los álbumes nunca están anidados. Dentro de cada álbum, las fotos se previsualizan en mosaico.

Cómo se construye — stack y arquitectura

PRODUCE

`plan.md`

- Stack técnico (frameworks, lenguajes, librerías)
- Arquitectura y patrones
- Modelo de datos
- Fases: Foundation → Core → Polish
- Check contra "constitution" del proyecto

CONSTITUTION

Principios no negociables del proyecto.

Spec Kit la introduce como archivo. Aplica en cada fase.

- "TDD obligatorio"
- "CLI-first en toda librería"
- "Sin SQL crudo, todo vía ORM"
- "Tipos estrictos, prohibido any"

El desglose atómico

PRODUCE

tasks.md

- Tareas atómicas (minutos, no días)
- Dependencias entre tareas
- Marcador [P] si paraleliza
- Rutas de archivo concretas
- Tests primero si aplica TDD

UNA TAREA QUEDA ASÍ

```
## Task 2.3 [P] · POST /albums
**Archivos:**

src/api/albums/create-album.controller.ts
src/api/albums/dto/create-album.dto.ts

**Depende de:** Task 2.1 (modelo)
                  Task 2.2 (service)

**Test primero:**
create-album.controller.spec.ts

**Criterios:**
· valida con Zod
· 201 con Location header
· 409 si nombre duplicado
```

FASE 04 · IMPLEMENT

El código, por fin.

Una tarea cada vez.

Diffs pequeños, enfocados. No dumps de mil líneas. El humano revisa cambios pequeños y aprueba.

Por fases. Validar antes de avanzar.

Implementación monolítica = contexto saturado = errores.

MÓDULO 05

SDD a mano

cuatro slash commands y nada más

Lo que necesitáis en el repo

Cuatro slash commands y una carpeta specs/. Cero dependencias externas.

```
mi-proyecto/  
├── CLAUDE.md  
├── specs/  
│   ├── 001-album-organizer/  
│   │   ├── spec.md  
│   │   ├── plan.md  
│   │   └── tasks.md  
│   ├── 002-photo-upload/  
│   │   └── ...  
└── .claude/  
    ├── commands/  
    │   ├── spec-new.md  
    │   ├── spec-plan.md  
    │   ├── spec-tasks.md  
    │   └── spec-implement.md
```


/spec-new — empezar una feature

```
---
```

```
description: Crea spec.md inicial para una feature nueva
```

```
argument-hint: <nombre-feature>
```

```
---
```

Estoy iniciando una feature nueva: \$ARGUMENTS

Crea la carpeta `specs/<NNN>-\$ARGUMENTS/` donde NNN es el siguiente número correlativo (mira las carpetas que ya existen).

Dentro, crea spec.md con: Contexto, User stories, Requisitos funcionales, Criterios de éxito, Casos frontera, Fuera de alcance.

Hazme las preguntas necesarias para rellenarlo.

NO incluyas stack técnico ni decisiones de implementación.

.claude/commands/spec-new.md

/spec-plan — del qué al cómo

```
---
description: Genera plan.md técnico para la spec actual
argument-hint: <numero-feature>
---
```

Lee `specs/\$ARGUMENTS-\*/spec.md` y CLAUDE.md (stack y convenciones).

Genera plan.md con:

- ## Stack técnico (versiones concretas)
- ## Arquitectura (patrón a aplicar, justificación)
- ## Modelo de datos (tablas, migraciones)
- ## Fases (Foundation → Core → Polish)
- ## Riesgos técnicos
- ## Check contra CLAUDE.md (¿viola algún anti-patrón?)

NO escribas código. Espera mi aprobación del plan.

.claude/commands/spec-plan.md

/spec-tasks — trocear el plan

```
---
```

```
description: Trocea el plan en tareas atómicas
```

```
argument-hint: <numero-feature>
```

```
---
```

Lee spec.md y plan.md de `specs/\$ARGUMENTS-\*/`.

Genera tasks.md con tareas atómicas:

- Cada tarea con un ID (Task 1.1, 1.2, 2.1, ...)
- Archivos exactos que toca
- Dependencias con otras tareas
- Marcador [P] si paraleliza con otras de su fase
- Test primero si aplica TDD

Una tarea = < 30 min de un humano. Si es más, trocéala.

`.claude/commands/spec-tasks.md`

/spec-implement — ejecutar tarea

```
---
```

```
description: Ejecuta una tarea de la spec actual
```

```
argument-hint: <numero-feature> <id-tarea>
```

```
---
```

```
Lee tasks.md y localiza la tarea $2.
```

```
Antes de tocar nada:
```

1. Comprueba que las dependencias estén [x] hechas.
2. Lee los archivos relacionados en el repo.
3. Si la tarea exige test primero, escríbelo y verifica que falla.

```
Implementa SOLO lo que la tarea pide.
```

```
NO toques archivos fuera de los listados.
```

```
Al terminar, marca la tarea como [x] en tasks.md.
```

POR QUÉ A MANO ANTES QUE TOOLING

Hacerlo a mano una vez os enseña qué hace cada fase y por qué.

Después podéis pasar a Spec Kit u otra herramienta sabiendo lo que automatiza.

MÓDULO 06

SDD con tooling

panorama 2026

GitHub Spec Kit

ORIGEN	GitHub · open source (sept 2025)
ESTRELLAS	~70k+ (mayo 2026)
COMANDO CLI	specify init <proyecto>
AGENTES	20+ (Claude Code, Copilot, Cursor, Gemini...)
DIFERENCIAL	constitution.md — principios inviolables

SLASH COMMANDS	
/speckit.constitution	<i>principios del proyecto</i>
/speckit.specify	<i>qué se construye</i>
/speckit.clarify	<i>resuelve ambigüedades</i>
/speckit.plan	<i>cómo se construye</i>
/speckit.tasks	<i>troceado atómico</i>
/speckit.analyze	<i>gaps cross-artefacto</i>
/speckit.implement	<i>ejecución</i>

Otras opciones con tracción

AWS Kiro

IDE completo con SDD nativo. AWS documenta features 40h → 8h. UI y steering persistente.

Tessl

Specs como contratos verificables. Más cercano a TLA+ en filosofía.

OpenSpec

Spec-first agnóstico, formato YAML/Markdown estandarizado.

BMAD

Agentes especializados por rol (Analyst, PM, Architect) que colaboran.

Google Antigravity

Versión de Google del enfoque spec-first integrada con sus herramientas.

cc-sdd

Plantillas estilo Kiro pero corriendo sobre Claude Code o Gemini CLI.

Cuál elegir según equipo

EQUIPOS PEQUEÑOS · UN PROYECTO

SDD a mano

Cuatro slash commands en Claude Code. Cero dependencias, todo en el repo, vosotros controláis el formato.

EQUIPOS MEDIANOS · VARIOS PROYECTOS

GitHub Spec Kit

El estándar de facto. Agnóstico de agente, comunidad grande, ecosistema de extensiones.

EQUIPOS GRANDES · COMPLIANCE

Spec Kit + constitution

Constitution fuerte y extensiones de auditoría. O Kiro si el equipo va all-in en su IDE.

MUCHAS REGENERACIONES

Tessl o BMAD

Cuanto más sofisticada la spec, más leverage da regenerar con cada modelo nuevo.

MÓDULO 07

Spec drift:

mantener la spec viva

Los tres tipos de drift

No es escribir mal la spec inicial. Es no actualizarla cuando el código se desvía.

01 Drift de implementación

El código diverge de la spec en silencio durante el implement.

Hook PostToolUse que avisa si un diff toca código sin referencia a una tarea.

02 Drift por cambio de requisitos

El producto cambia, se actualiza el código, no la spec.

PR rule: si el código cambia comportamiento, la spec debe cambiar en el mismo PR.

03 Drift cruzado

Dos specs se contradicen tras meses de evolución.

/speckit.analyze periódico o slash command propio equivalente.

LA REGLA DE ORO

Si el código que vas a escribir
no se desprende lógicamente
de algo escrito en la spec,

no es código: es deuda técnica con disfraz.

Actualiza la spec antes.

Checklist antes de cerrar la feature

☐

¿Cada requisito de la spec está cubierto por al menos una tarea?

☐

¿Cada tarea está [x] cerrada o reescalada?

☐

¿Los tests cubren criterios de éxito de la spec, no solo detalles de implementación?

☐

¿Los anti-patrones del CLAUDE.md siguen respetados?

☐

¿La spec sigue siendo legible en 6 meses por alguien que se incorpora hoy?

LO QUE DICE LA INDUSTRIA

"El moat no son las specs.

Es el leverage de regeneración

que las specs maduras te dan con cada nueva generación de modelos."

Los equipos que empezaron a construir su biblioteca de specs en 2025 o 2026 pueden regenerar contra Claude 6, GPT-6 o lo que venga.

Los que se retrasaron, no.

TRAMO FINAL

Cierre

lo que importa · deberes · preguntas

Tres ideas para la próxima sesión

01 La spec es la fuente de verdad, no el código

Es el cambio mental de SDD. El código se regenera; la spec se conserva y se versiona como cualquier artefacto crítico.

02 Cuatro fases, cuatro artefactos

Specify → Plan → Tasks → Implement. Cada uno produce un .md que alimenta al siguiente. No se salta uno.

03 Empieza a mano, adopta tooling cuando duela

Cuatro slash commands propios os llevan al 80%. Spec Kit u otras herramientas son el siguiente paso, no el primero.

Deberes para la sesión 3

-  **01** Crear specs/ en vuestro repo y los 4 slash commands (spec-new, spec-plan, spec-tasks, spec-implement)
-  **02** Escribir una spec completa para una feature pequeña real (1-2 días, no un toy example)
-  **03** Llegar hasta tasks.md aprobado sin escribir código todavía
-  **04** Implementar al menos 3 tareas con /spec-implement y observar diffs
-  **05** Instalar GitHub Spec Kit en una rama experimental, comparar con vuestra versión "a mano"
-  **06** Traer una pieza de código generado vía SDD que no os convenza del todo

LO QUE VIENE

Sesión 3

Revisión de código y refactorización

Pasamos del "generar bien" al "revisar bien lo que ya hay". Code review asistido, detección de code smells, refactor con criterio, generación de tests sobre código legacy, y cómo extraer specs retrospectivamente.

Preguntas *y debate.*

VALEN · VA360 LABS